



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Παραλληλα Συστήματα
Εργασία 2

Ονοματεπώνυμο: ΠΑΚΙΤΣΑΚ ΝΤΕΛΙΑ-ΜΑΡΙΑ /ΙΑΣΩΝ ΚΑΡΑΠΡΟΔΡΟΜΙΔΗΣ
Αριθμός Μητρώου: 1115202200125 / 1115202200064

Περιεχόμενα

Εισαγωγή	2
1 Ασκήση 1	2
1.1 Περιγραφή Προβλήματος	2
1.2 Περιγραφή Λύσης	2
1.3 Πειραματικά αποτελέσματα και σχολιασμός	3
2 Ασκήση 2	6
2.1 Περιγραφή Προβλήματος	6
2.2 Περιγραφή Λύσης	6
2.3 Πειραματικά αποτελέσματα και σχολιασμός	6
3 Ασκήση 3	13
3.1 Περιγραφή Προβλήματος	13
3.2 Περιγραφή Λύσης	13
3.3 Πειραματικά Αποτελέσματα και Σχολιασμός	14

Εισαγωγή

Για την παραγωγή γραφημάτων χρησιμοποιήσαμε *script* σε *python*. Οι ασκήσεις εκτελέστηκαν στα μηχανήματα *linux* της σχολής τα οποία έχουν 4 πυρήνες και 1 νημά ανά πυρήνα. Οι χρόνοι στα πειράματα είναι ο μέσος όρος από 4 εκτελέσεις. Χρησιμοποιήσαμε *cmake* για το *build* διότι εκμεταλλευτήκαμε τη δύναμη του για να αλλάζουμε γρήγορα ρυθμίσεις στη μεταγλωττίση χωρίς να χρειάζεται να κάνουμε πολλά, όπως η μεταγλωττίση χωρίς *OpenMP*, η οποία χρειάστηκε συχνά για την μέτρηση επίδοσης των σειριακών εκδόσεων των προγραμμάτων.

1 Ασκήση 1

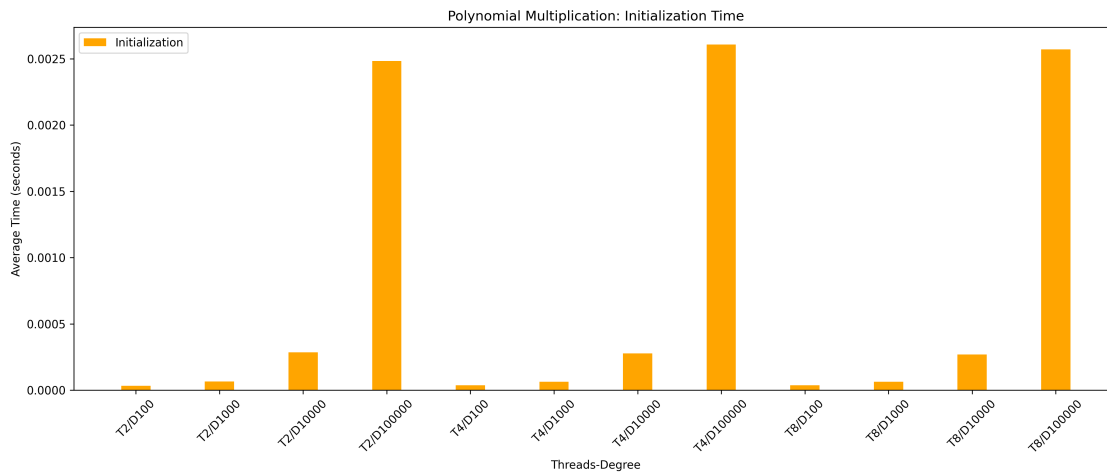
1.1 Περιγραφή Προβλήματος

Η άσκηση ζητά να υλοποιήσουμε ένα πρόγραμμα που υπολογίζει τον πολλαπλασιασμό δυο τυχαίων πλήρων πολυωνύμων βαθμού n με μη μηδενικούς ακεραίους συντελεστές τόσο με σειριακό όσο και με παραλληλό αλγόριθμο χρησιμοποιώντας *OpenMP*. Στην συνέχεια θα πρέπει να μετρήσουμε και να συγκρίνουμε τους χρόνους εκτέλεσης για την αρχικοποίηση του προβλήματος, του σειριακού υπολογισμού και του παραλληλού υπολογισμού καθώς και να συγκρίνουμε τα αποτελέσματα του παραλληλού υπολογισμού με *OpenMP* έναντι του παραλληλού υπολογισμού με *Pthreads* (εργασία 1). Τέλος θα πρέπει να επιβεβαιώσουμε ότι τα αποτελέσματα των δυο αλγορίθμων (σειριακό/παραλληλό) είναι ίδια και να μελετήσουμε την απόδοση για διαφορετικό αριθμό νημάτων και διαφορετικό αριθμό βαθμού πολυωνύμων

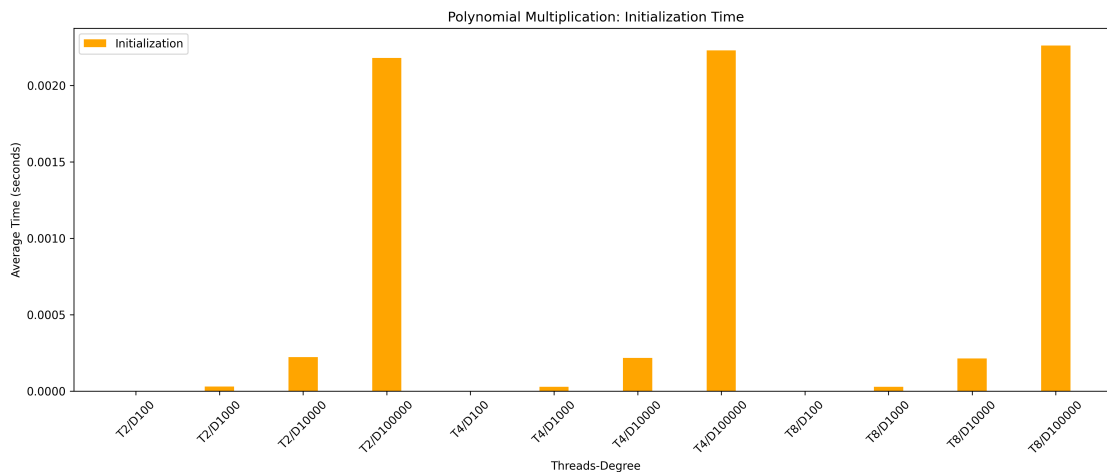
1.2 Περιγραφή Λύσης

Αρχικά το πρόγραμμα ελέγχει αν τα ορίσματα που δέχεται είναι σωστά, δηλαδή αν δόθηκαν τιμές για τον βαθμό των πολυωνύμων και τον αριθμό των *thread*. Δημιουργούμε δυναμικά τους πίνακες που θα περιέχουν τους τυχαίους μη μηδενικούς συντελεστές. Στην συνέχεια αφού δημιουργήσουμε τους τυχαίους συντελεστές με την χρήση της συνάρτησης *generate_polynomials* και μετρήσουμε τον χρόνο αρχικοποίησης θα εκτελέσουμε τον σειριακό πολλαπλασιασμό με διπλή επαναληψη όπως κάναμε στην εργασία 1 (ασκήση 1.1). Ο κάθε συντελεστής του αποτελέσματος προκύπτει από το άθροισμα γινομένων των συντελεστών των δυο πίνακων που περιέχουν τους συντελεστές των πολυωνύμων. Αντιστοίχα για τον παραλληλό αλγόριθμο χρησιμοποιούμε την οδηγία *pragma omp parallel for* με την επιλογή *schedule(static)* η οποία αναθέτει στα νημάτα έναν συγκεκριμένο αριθμό της επαναληψής *for*. Με αυτόν τρόπο κάθε νημά θα αναλαμβάνει να υπολογίζει ένα ανεξάρτητο τμήμα του πίνακα, δηλαδή σε κάθε επαναληψη θα υπολογίζουμε το άθροισμα των γινομένων για κάθε συντελεστή $(x, x^2, x^3, \dots)$. Για να μην χρειαστεί να χρησιμοποιήσουμε *mutex-atomic* χρησιμοποιούμε 2 μεταβλητές *start* και *end* που θα διασφαλίζουν ότι για κάθε βαθμό του συντελεστή θα χρησιμοποιούμε μόνο τα στοιχεία του πίνακα που χρειζόμαστε. Για παράδειγμα αν έχουμε δυο πολυώνυμα 2ου βαθμού ($n=2$) και χρησιμοποιούμε 2 νημάτα το αποτέλεσμα θα έχει 5 συντελεστές $(x_0 - x_4)$. Το πρώτο νημά θα αναλάβει τους βαθμούς $i = 0, 1, 2$ και το δεύτερο τους $i = 3, 4$. Για τον συντελεστή $i = 1$ οι μεταβλητές θα οριστούν ως *start* = 0 και *end* = 1 οδηγώντας στον υπολογισμό του αθροίσματος $(P1[0] * P2[1]) + (P1[1] * P2[0])$. Αντιστοίχα για τον συντελεστή $i = 4$ το δεύτερο νημά θα θέσει *start* = 2 και *end* = 2 υπολογίζοντας το γινόμενο $P1[2] * P2[2]$. Με αυτόν τον τρόπο επειδή το νημά θα γράφει κάθε φορά σε διαφορετική θέση του πίνακα αποτελεσμάτων οι υπολογισμοί μπορούν να γίνουν παραλληλά χωρίς κίνδυνος ταυτοχρόνης εγγραφής στην ίδια μνήμη. Μετά την ολοκλήρωση των αλγορίθμων συγκρίνουμε τα αποτελέσματα του σειριακού αλγορίθμου και του παραλληλού αλγορίθμου για επιβεβαίωση ορθότητας.

1.3 Πειραματικά αποτελέσματα και σχολιασμός

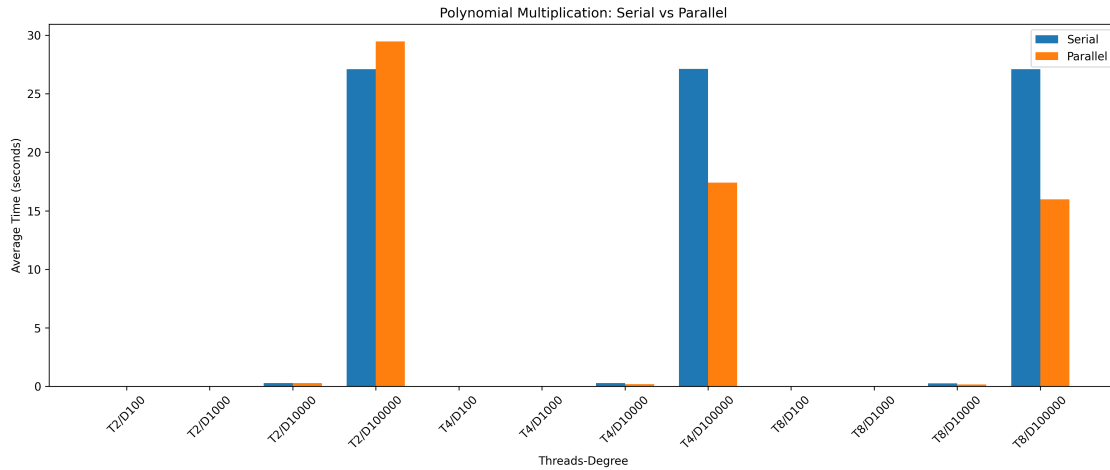


(α') Initialization time for the exercise 1.1 (previous homework)

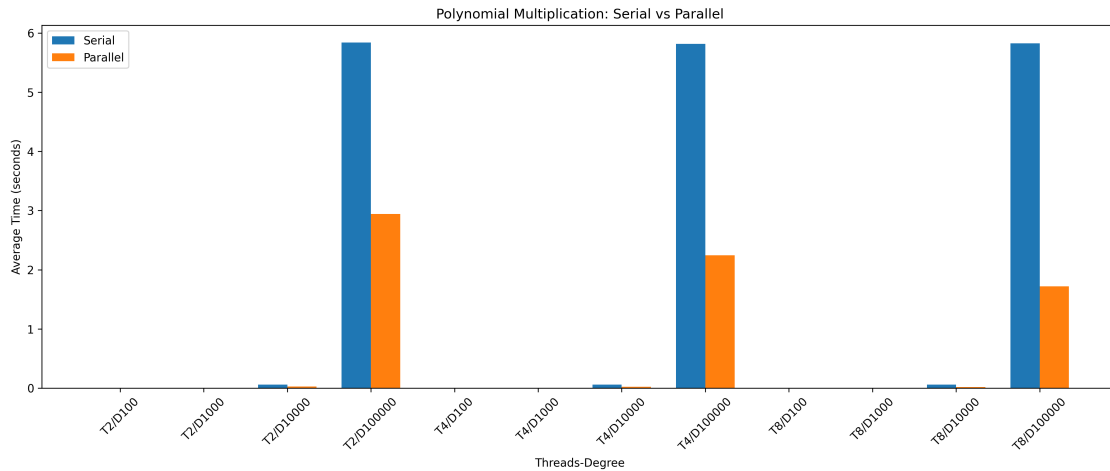


(β') Initialization time for the exercise 2.1 (current homework)

Όπως παρατηρούμε από τα πειραματικά αποτελέσματα που συλλεχθήκαν από το *script.py* η απόδοση του παραλλήλου αλγορίθμου εξαρτάται έντονα από τον βαθμό του πολυωνύμου και τον αριθμό των *threads*. Με βάση τα παραπάνω γραφήματα της άσκησης 1.1 και άσκηση 2.1 παρατηρούμε ότι ο χρόνος αρχικοποίησης (*initialization time*) έχουν την ίδια αυξητική τάση και στις δύο υλοποιήσεις (*Pthreads* και *OpenMP*) ενώ αυξάνεται ο βαθμός του πολυωνύμου διότι πρέπει να παραπαγούμε περισσότερα στοιχεία. Επίσης βλέπουμε ότι οι χρόνοι είναι πολύ παρόμοιοι. Αυτό συμβαίνει διότι η διαδικασία της αρχικοποίησης (δεσμεύση μνήμης και παραγωγή τυχαίων συντελεστών) εκτελείται με τον ίδιο τρόπο και στα δύο προγράμματα πριν από την έναρξη των αλγορίθμων.



(α') Graphic bars for serial vs parallel algorithm for the exercise 1.1 (previous homework)



(β') Graphic bars for serial vs parallel algorithm for the exercise 2.1 (current homework)

Με βάση τα αποτελέσματα των μετρήσεων και των δύο υλοποιήσεων παρατηρούμε ιδιαίτερη επιτάχυνση *speedup* όσο αυξάνεται ο βαθμός του πολυωνυμίου. Για μικρούς βαθμούς (π.χ. 100) ο σειριακός αλγόριθμος είναι ταχύτερος καθώς το κόστος δημιουργίας και συγχρονισμού των νημάτων υπερβαίνει το χρόνο που κερδίζουμε από την παραλληλή εκτέλεση. Πιο συγκεκριμένα όταν φτίνουμε πολλά *threads* το λειτουργικό σύστημα πρέπει να τα δημιουργήσει και να τα συγχρονίσει καθώς η "ΠΥ" δαπανά περισσότερους κύκλους ρολογιού για το *context switching*. Αυτός ο χρόνος δεν συλλαμβάνεται άμεσα στον υπολογισμό αλλά είναι απαραίτητος για να λειτουργήσει η παραλληλία. Αντίθετα, για μεγάλους βαθμούς (π.χ. 10^5) η παραλληλή εκτέλεση είναι πιο αποδοτική που φτάνει έως και $3.39x(openMP)$ σε σχέση με τον σειριακό, όπως φαίνεται στον πίνακα. Αυτό συμβαίνει επειδή η υπολογιστική πολυπλοκότητα αυξάνεται εκθετικά με την αύξηση του βαθμού του πολυωνυμίου.

Η επιτάχυνση που παρατηρείται οφείλεται στο γεγονός ότι ο αλγόριθμος επιτρέπει σε κάθε *thread* να επεξεργάζεται ανεξάρτητο τμήμα του πίνακα αποτελεσμάτων αποφεύγοντας συγχρονισμούς και αναγκάζει για συγχρονισμό. Δηλαδή κάθε *thread* αναλαμβάνει μια επαναληψη η οποία υπολογίζει το γινόμενο για έναν συγκεκριμένο βαθμό ($x, x^2, \dots$). Για αυτόν τον λόγο δεν χρησιμοποιήθηκαν *mutex* ή *atomics* για την προστασία κάποιας κοινής μεταβλητής αφού δεν υπήρχε κοινή πρόσβαση σε ίδιες θέσεις μνήμης. Η έλλειψη χρήσης συγχρονισμού είναι πολύ σημαντική για την υψηλή απόδοση στους παραλληλούς αλγόριθμους αφού επειδή κάθε νήμα έχει πρόσβαση σε διαφορετικές θέσεις μνήμης επιτρέπει στους πυρήνες της *CPU* να λειτουργούν στη

μεγιστη ταχυτητα χωρις καθυστερησεις αναμονης.

Summary of Average Times and Speedup

Threads-Degree	Serial	Parallel	Improvement (Serial/Parallel)
T2/D100	0.000035	0.000358	0.10x
T2/D1000	0.003025	0.003409	0.89x
T2/D10000	0.280167	0.293453	0.95x
T2/D100000	27.100279	29.463178	0.92x
T4/D100	0.000031	0.000327	0.09x
T4/D1000	0.003020	0.002559	1.18x
T4/D10000	0.273830	0.183713	1.49x
T4/D100000	27.113639	17.409730	1.56x
T8/D100	0.000033	0.000452	0.07x
T8/D1000	0.003015	0.001965	1.53x
T8/D10000	0.271753	0.163021	1.67x
T8/D100000	27.096784	15.989303	1.69x

(α') Table with the exact times for the exercise 1.1 (previous homework)

Summary of Average Times and Speedup

Threads-Degree	Serial	Parallel	Improvement (Serial/Parallel)
T2/D100	0.000007	0.000068	0.10x
T2/D1000	0.000709	0.000866	0.82x
T2/D10000	0.058925	0.029389	2.00x
T2/D100000	5.838054	2.940614	1.99x
T4/D100	0.000007	0.000281	0.03x
T4/D1000	0.000670	0.000350	1.91x
T4/D10000	0.059093	0.023045	2.56x
T4/D100000	5.816308	2.244304	2.59x
T8/D100	0.000008	0.000253	0.03x
T8/D1000	0.000668	0.000521	1.28x
T8/D10000	0.059439	0.020835	2.85x
T8/D100000	5.822360	1.719966	3.39x

(β') Table with the exact times for the exercise 2.1 (current homework)

Συμπερασματικά η παραλληλη υλοποιηση με *openMP* του αλγοριθμου πολλαπλασιασμου πολυωνυμων παρουσιαζει σημαντικη επιταχυνση για μεγάλους βαθμους και αυξημενο αριθμο νηματων. Η αποδοση βελτιωνεται καθως αυξανεται το μεγεθος του προβληματος. Η πολυπλοκοτητα του αλγοριθμου ειναι $O(n^2)$ καθως για καθε συντελεστη του πρωτου πολυωνυμου γινονται πολλαπλασιασμοι με ολους τους συντελεστες του δευτερου. Για παραδειγμα αν εχουμε βαθμο 10^5 ο συνολικος αριθμος πραξεων ειναι 10^{10} . Ενας σειριακος αλγοριθμος πρεπει να εκτελεσει ολες αυτες τις πραξεις διαδοχικα, ενω στην παραλληλη εκτελεση οι πραξεις μοιραζονται σε πολλα *threads* μειωνοντας τον πραγματικο χρονο εκτελεσης.

Συγκρινοντας την αποδοση του *OpenMP* (ασκηση 2.1) με τα *Pthreads* (ασκηση 1.1) παρατηρουμε διαφορες που πηγάζουν απο την αρχιτεκτονικη των δυο μοντελων. Τα *Pthreads* ειναι μια *low – level* βιβλιοθηκη οπου θα πρεπει εμεις οι ιδιοι να εχουμε την ευθυνη για τη διαχειριση καθε νηματος (δημιουργια, δρομολογηση, συγχρονισμος) με αποτελεσμα για βαθμο $n = 100.000$ με 8 νηματα να επιτυγχανουμε βελτιωση μολις $1.69x$ (απο $27.09s$ σε $15.98s$). Αντθετα το *OpenMP* λειτουργει σαν μια *high – level* διεπαφη κρυβοντας την πολυπλοκοτητα της διαχειρισης των νηματων πισω απο απλες οδηγιες. Επιτρεπει στον *compiler* να εφαρμοσει αυτοματες βελτιστοποιησεις πετυχαινοντας για το ιδιο μεγεθος προβληματος βελτιωση $3.39x$ δηλαδη σχεδον διπλασια επιταχυνση απο το *Pthreads*. Η τεραστια αυτη διαφορα στην αποδοση οφειλεται κυριως στη βελτιστοποιημενη διαχειριση των επαναληψεων στην εκδοση του *OpenMP*. Ενω στα *Pthreads* ο διαμοιρασμος ηταν στατικος και η εσωτερικη λουπα περιελαμβανε ελεγχους *if* στην υλοποιηση του *OpenMP* χρησιμοποιηθηκαν δυναμικα ορια (*start* και *end*) περιοριζοντας τις πραξεις μονο στις απολυτως απαιτητες και αποφευγοντας περιττους κυκλους στη *CPU*.

2 Ασκήση 2

2.1 Περιγραφή Προβλήματος

Έπρεπε να υλοποιήσουμε ένα πρόγραμμα που πραγματοποιεί πολλαπλασιασμό πίνακα με διάνυσμα σε δύο διαφορετικές μορφές: μία με πυκνό πίνακα, και μία με τη μορφή CSR.

Η μορφή αυτή χωρίζει τον Matrix σε τρεις πίνακες: 1. Έναν με τα μη μηδενικά στοιχεία. 2. Έναν με το index της στήλης στην οποία βρίσκεται το κάθε στοιχείο. 3. Έναν που κρατάει τον αριθμό των μη μηδενικών στοιχείων πριν από κάθε σειρά.

Με αυτόν τον τρόπο, μπορούμε να εκτελούμε τον πολλαπλασιασμό πολύ πιο γρήγορα σε αραιούς πίνακες, καθώς αποφεύγουμε την επεξεργασία των μηδενικών στοιχείων. Στη συνέχεια, έπρεπε να παραλληλοποιήσουμε το πρόγραμμα με τη χρήση του OpenMP και να συγκρίνουμε τα αποτελέσματα για διάφορες παραμέτρους.

2.2 Περιγραφή Λύσης

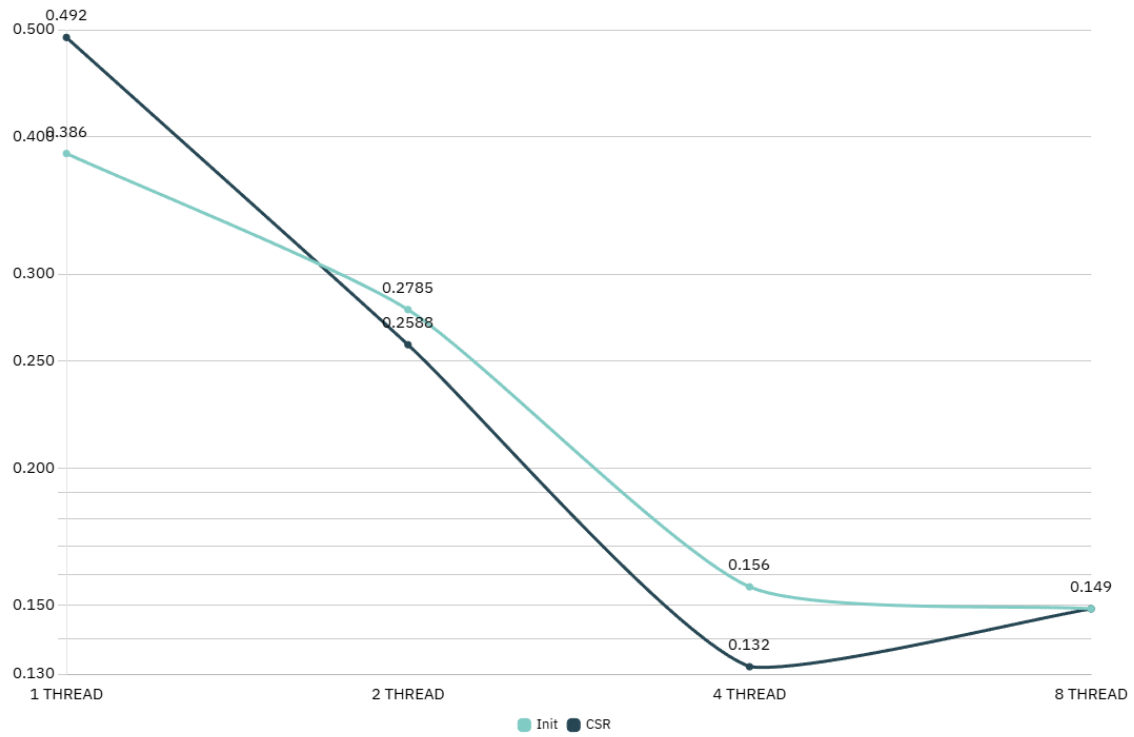
Για τους πίνακες χρησιμοποίησα μια μονοδιάστατη αναπαράσταση. Με αυτόν τον τρόπο μου ήταν πιο εύκολο να μετατρέψω τον πίνακα στη μορφή που θέλαμε. Για την πυκνή αναπαράσταση, τροποποίησα τον κώδικα του βιβλίου ώστε να δουλεύει με τα δεδομένα του προγράμματος. Για την CSR αναπαράσταση, πρώτα μετέτρεψα τα δεδομένα στη μορφή των 3 πινάκων και μετά υλοποίησα τον αλγόριθμο για τον πολλαπλασιασμό.

Η παραλληλοποίηση του κώδικα ήταν αρκετά εύκολη, γεγονός που αποτελεί και τη δύναμη του OpenMP, καθώς δεν απαιτήθηκαν ιδιαίτερες τροποποιήσεις, εκτός από το κομμάτι της μετατροπής του πυκνού πίνακα σε CSR. Σε εκείνο το σημείο υπήρχαν εξαρτήσεις σχετικά με τη θέση στην οποία θα έμπαιναν τα στοιχεία στους πίνακες των columns και των μη μηδενικών στοιχείων.

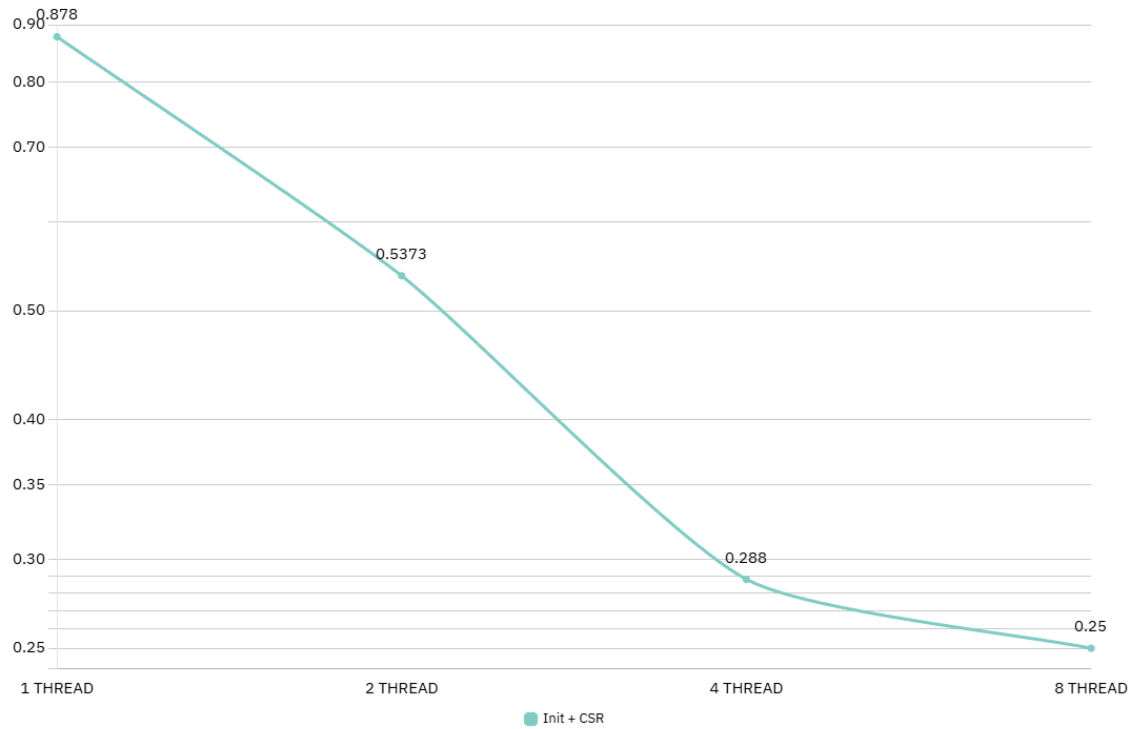
Για να εξαλείψω αυτή την εξάρτηση, έπρεπε να χωρίσω τη διαδικασία σε δύο μέρη: στη συλλογή των δεδομένων και έπειτα στη μετατροπή των πινάκων στην τελική μορφή. Αρχικά συγκέντρωσα τα στοιχεία σε διδιάστατους πίνακες, όπου κάθε νήμα αναλάμβανε μια γραμμή και τοποθετούσε τα στοιχεία σε προσωρινούς πίνακες. Το σημαντικό ήταν ότι πλέον γνωρίζαμε την ακριβή θέση στους πίνακες V και colIdx όπου έπρεπε να μπει τα στοιχεία, οπότε και η μετατροπή μπορούσε να γίνει παράλληλα. Με αυτόν τον τρόπο παραλληλοποίησα όλους τους αλγόριθμους και ξεκίνησα τις μετρήσεις. Εν τέλει, δεν χρειάστηκε συγχρονισμός σε κανένα σημείο του προγράμματος.

2.3 Πειραματικά αποτελέσματα και σχολιασμός

Για αρχή έκανα πειράματα για να δω πόσο καλά παραλληλοποιείται η CSR αναπαράσταση του προγράμματος. Χρησιμοποίησα το default schedule, διότι τα άλλα απλά έριχναν την απόδοση του προγράμματος. Η επιλογή του default schedule (που συνήθως είναι το static) είναι συχνά η καλύτερη για προβλήματα με ομοιόμορφο φόρτο εργασίας, καθώς έχει το χαμηλότερο overhead. Αλλά σχήματα (όπως το dynamic) έριχναν την απόδοση, αυτό οφείλεται πιθανότατα στον επιπλέον χρόνο που χρειάζεται το σύστημα για τη διαχείριση των νημάτων (scheduling overhead).



(α') Parallel scaling breakdown: Execution time for individual components (Initialization vs. Multiplication).

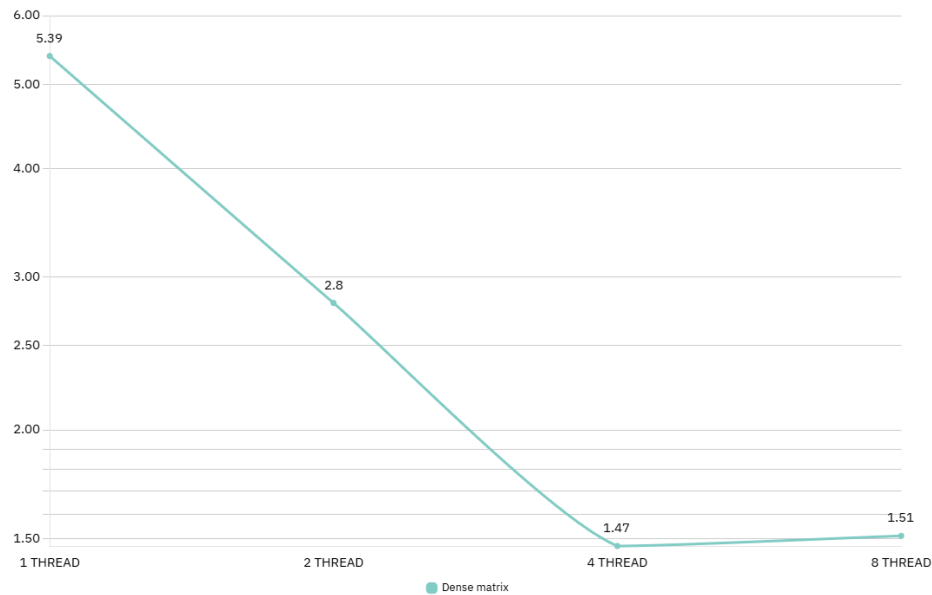


(β') Combined parallel scaling: Total execution time (Initialization + CSR) across thread counts.

Σχήμα 4: Parallel performance analysis: (a) highlights the scalability of the multiplication kernel versus the sequential nature of initialization, while (b) shows the aggregate impact on total runtime.

Κοιτάζοντας το γράφημα, βλέπουμε ότι το πρόγραμμα κάνει πολύ καλό scale, διότι δεν υπάρχει ανάγκη συγχρονισμού και υπάρχει πολλή δουλειά που μπορούν να κάνουν τα νήματα. Η επιτάχυνση φαίνεται να είναι πολύ παρόμοια μεταξύ της αρχικοποίησης και του πολλαπλασιασμού, παρότι η παράλληλη έκδοση της αρχικοποίησης είναι λιγότερο efficient.

Ωστόσο, μετά από 4 threads η απόδοση μένει σχεδόν ίδια. Αυτό συμβαίνει διότι το συγκεκριμένο πρόβλημα είναι memory-bound, που σημαίνει ότι την περισσότερη ώρα την ξοδεύει το πρόγραμμα περιμένοντας να έρθουν τα δεδομένα από τη RAM στην cache. Οπότε, δεν υπάρχει λόγος να έχουμε παραπάνω νήματα, καθώς ο χρόνος δεν θα βελτιωθεί.

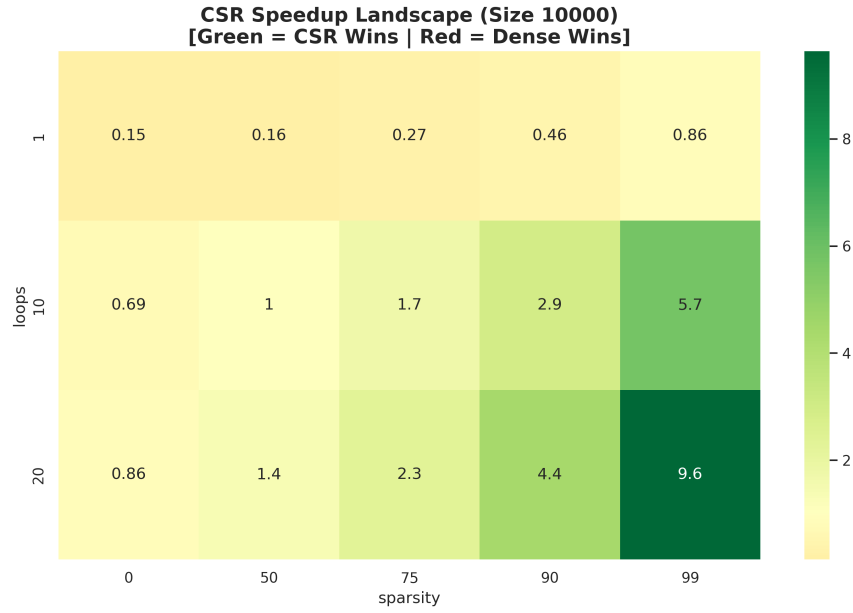


Σχήμα 5: Parallel scaling graph

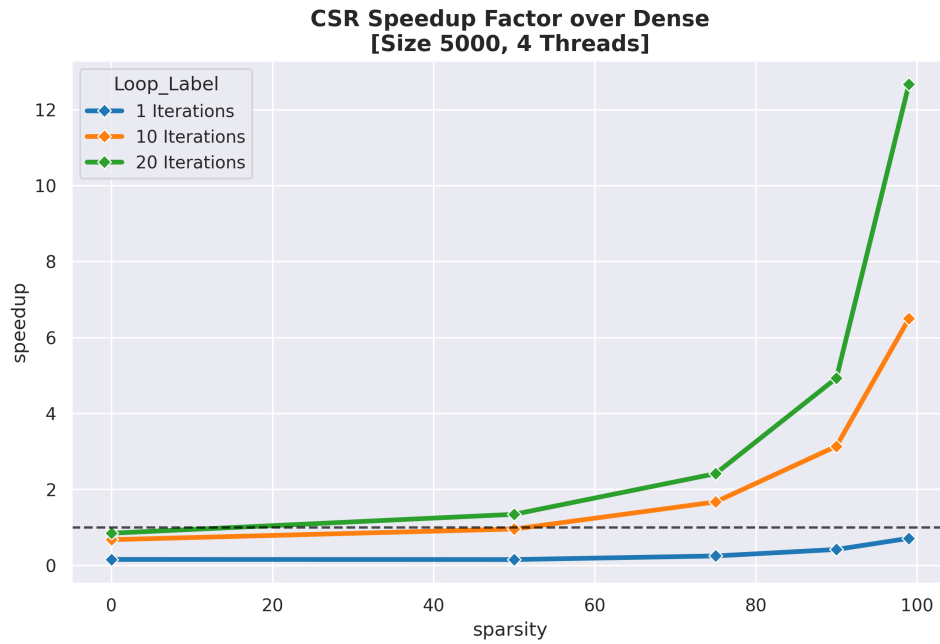
Κοιτάζοντας και την πυκνή αναπαράσταση, βλέπουμε ακριβώς την ίδια εικόνα: πολύ καλή παραλληλοποίηση μέχρι τα 4 νήματα.

Πράγματι, καθώς ο αλγόριθμος πολλαπλασιασμού είναι αρκετά παρόμοιος για τις δύο αναπαραστάσεις, είναι λογικό να βλέπουμε παρόμοια αποτελέσματα. Και οι δύο αναπαραστάσεις κάνουν πολύ καλό scale. Επίσης, επειδή υπάρχει αρκετή δουλειά, το overhead της αρχικοποίησης των νημάτων του OpenMP δεν είναι καθόλου αισθητό.

Σε σύγκριση με την πυκνή μορφή, η αναπαράσταση CSR, μετά από πειράματα, φαίνεται να είναι πιο γρήγορη όταν οι πίνακες έχουν περισσότερα μηδενικά στοιχεία και όταν κάνουμε παραπάνω επαναλήψεις. Όσο πιο αραιοί είναι οι πίνακες, τόσο μεγαλύτερο speedup έχει η αναπαράσταση CSR, όπως δείχνει το παρακάτω heatmap:



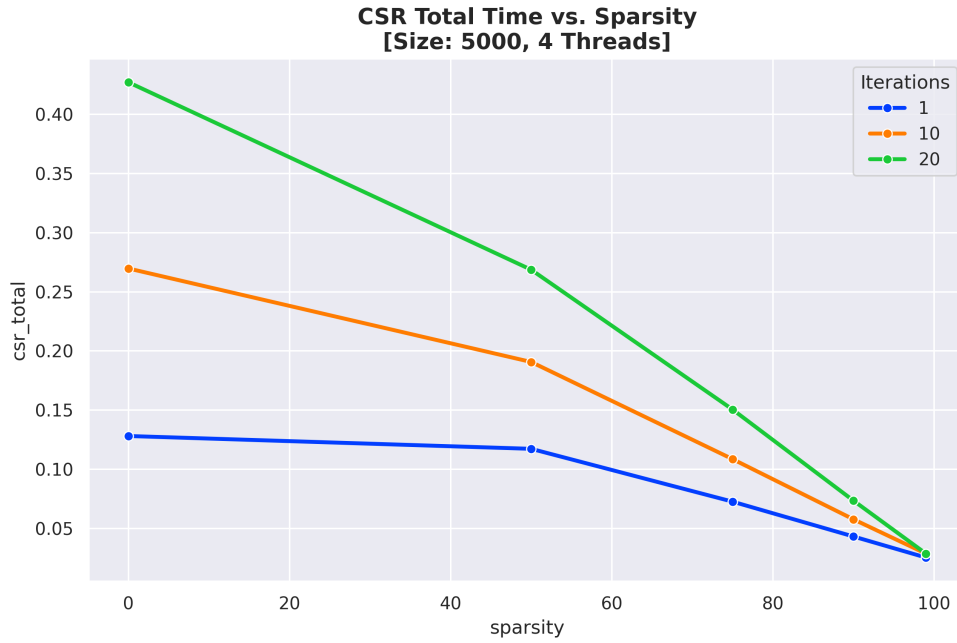
(α') Heatmap analysis showing the efficiency landscape of CSR across various iteration counts and sparsity levels.



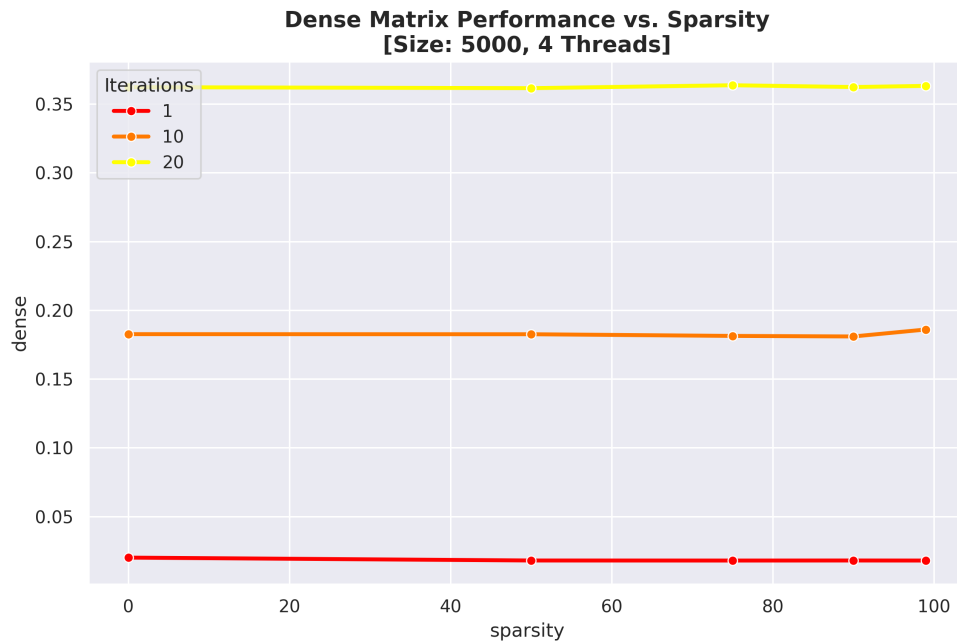
(β') Measured speedup ratio of the CSR implementation relative to the dense baseline as a function of sparsity.

Σχήμα 6: Comprehensive efficiency analysis: (a) provides a holistic view of performance gains (green regions) and losses (red regions), while (b) quantifies the acceleration factor achieved as the matrix becomes increasingly sparse.

Όπως δείχνει αυτό το heatmap, η dense αναπαράσταση, όπως και θα περιμέναμε, δεν κάνει καλά scale με περισσότερα μηδενικά στοιχεία. Όπως δείχνουν επίσης και τα παρακάτω γραφήματα:



(α') Total execution time for CSR format across sparsity levels.



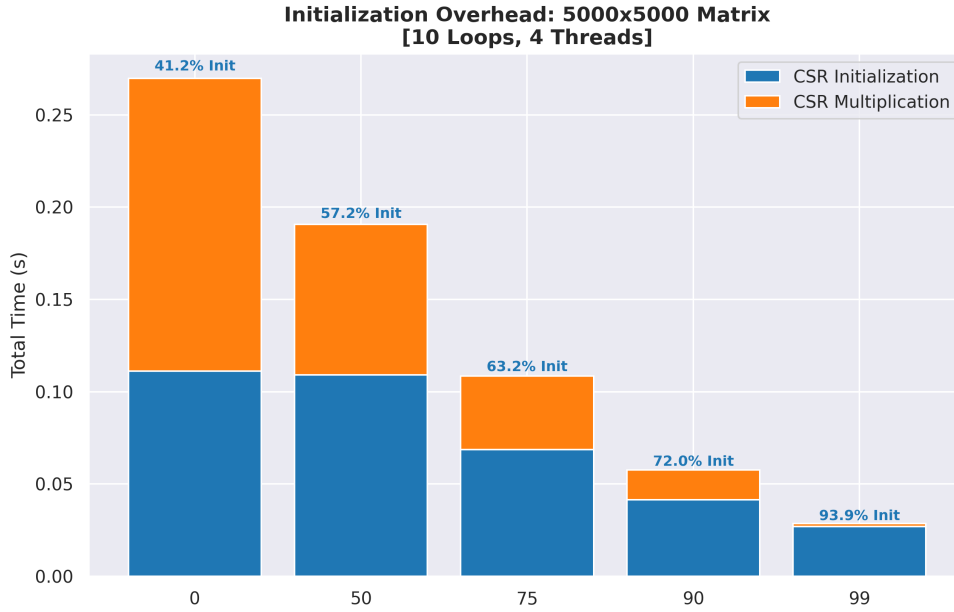
(β') Total execution time for Dense format across sparsity levels.

Σχήμα 7: Detailed computational performance analysis: (a) highlights the sensitivity of CSR to non-zero density, whereas (b) illustrates the overhead of the dense algorithm in sparse contexts.

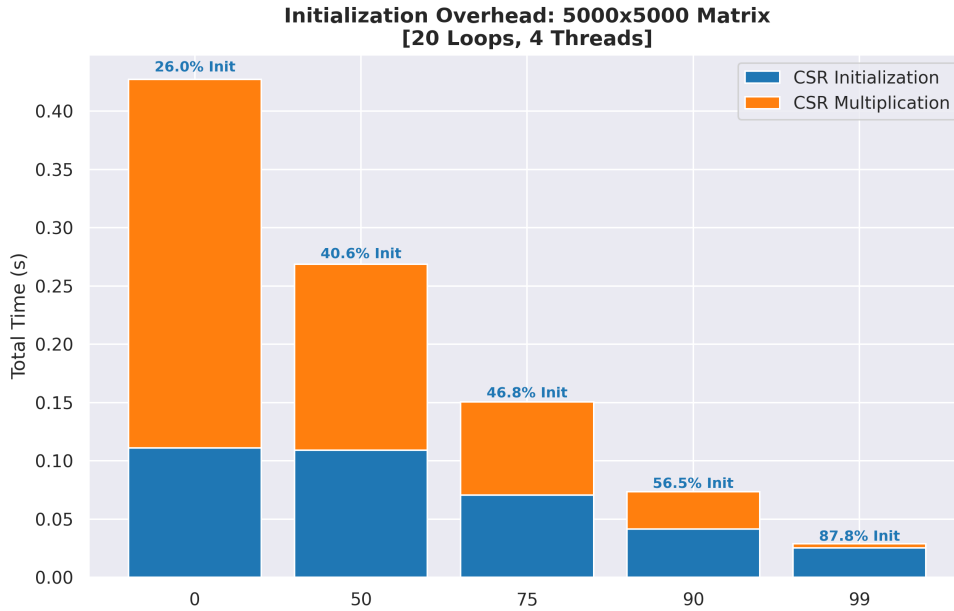
Ο χρόνος παραμένει σταθερός για την πυκνή αναπαράσταση, ενώ για τη μορφή CSR ο χρόνος μειώνεται όσο αυξάνεται η αραιότητα των πινάκων.

Το κόστος αρχικοποίησης των τριών πινάκων της αναπαράστασης CSR είναι ιδιαίτερα σημαντικό όταν ο αριθμός των επαναλήψεων είναι περιορισμένος, αλλά το σχετικό overhead αποσβένεται καθώς οι επαναλήψεις

αυξάνονται. Γενικότερα, όσο μεγαλύτερος είναι ο όγκος των υπολογισμών που αποφεύγουμε, τόσο περισσότερο δικαιολογείται το υπολογιστικό κόστος της μετατροπής από την `dense` μορφή. Η προσέγγιση αυτή κρίνεται ιδανική για επαναληπτικές μεθόδους, όπως τα προβλήματα σταθερού σημείου.



(α') Initialization overhead for 10 iterations (5000×5000 matrix).

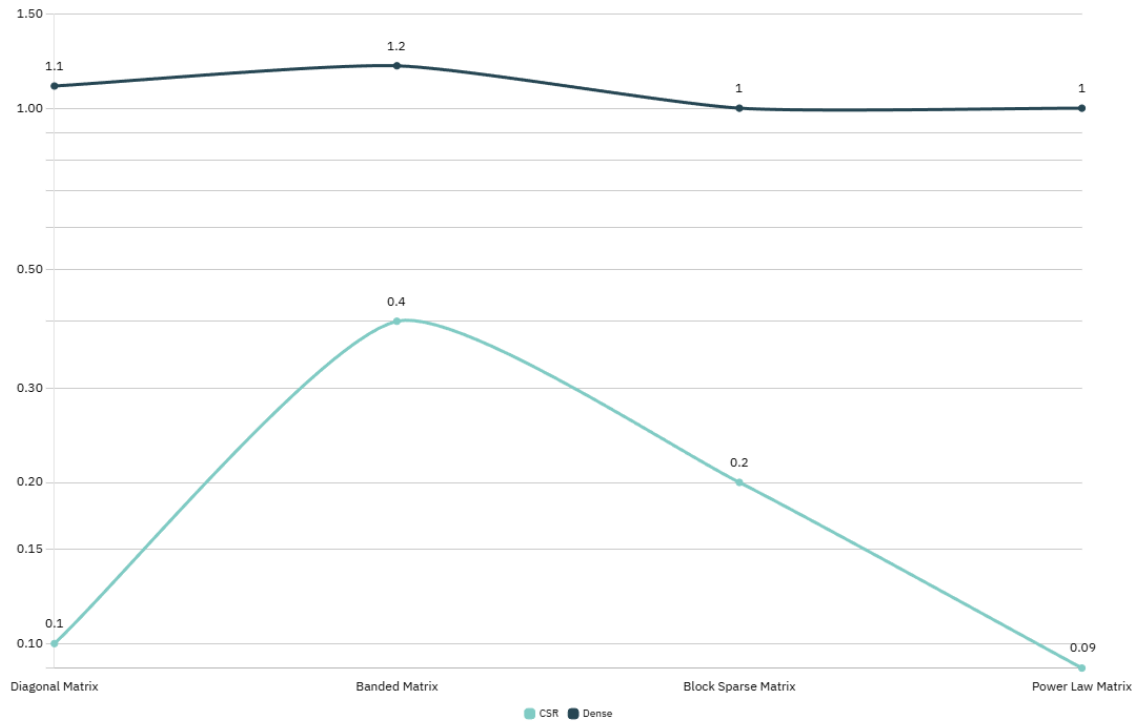


(β') Initialization overhead for 20 iterations (5000×5000 matrix).

Σχήμα 8: Analysis of CSR initialization overhead: The figures illustrate how the cost of building the CSR structure is amortized over an increasing number of multiplication iterations (10 vs. 20).

Τέλος, εξετάστηκαν ορισμένες ειδικές περιπτώσεις πινάκων που εμφανίζονται συχνά σε πραγματικές εφαρ-

μογές, ώστε να αξιολογηθεί η επίδοση της αναπαράστασης CSR σε ρεαλιστικά σενάρια. Τα πειράματα αυτά πραγματοποιήθηκαν σε προσωπικό υπολογιστή (σε αντίθεση με τις προηγούμενες μετρήσεις που έγιναν στα εργαστήρια της σχολής). Σημειώνεται ότι οι χρόνοι που παρουσιάζονται για τη μορφή CSR συμπεριλαμβάνουν και το κόστος της αρχικοποίησης:



Σχήμα 9: Special cases

Όπως παρατηρείται, επιτυγχάνεται εξαιρετικά υψηλή επιτάχυνση (speedup), ιδιαίτερα στην περίπτωση των διαγώνιων πινάκων. Αυτό εξηγείται από το γεγονός ότι η μορφή CSR μειώνει την υπολογιστική πολυπλοκότητα στο $O(n)$, εξαλείφοντας πλήρως την ανάγκη επεξεργασίας του τεράστιου όγκου μηδενικών στοιχείων που χαρακτηρίζει αυτούς τους πίνακες.

3 Ασκήση 3

3.1 Περιγραφή Προβλήματος

Το συγκεκριμένο πρόβλημα μας ζητάει να υλοποιήσουμε την ταξινόμηση *MergeSort* σειριακά και παράλληλα με την χρήση εντολών *OpenMP*. Το πρόγραμμα θα πρέπει να παράγει έναν τυχαίο πίνακα ακέραιων μεγεθους n που δίνεται από το όρισμα επιτρέποντας στο χρήστη να διαλέξει μεταξύ σειριακής ή παράλληλης υλοποίησης της ταξινόμησης *MergeSort* και τον αριθμό νημάτων που θα χρησιμοποιήσουμε στο πρόγραμμα. Τέλος θα πρέπει να μετρήσουμε και να μελετήσουμε τον χρόνο υλοποίησης που θα τυπώνουμε καθώς και να ελεγχούμε αν τα αποτελέσματα μας είναι ταξινομημένα σωστά .

3.2 Περιγραφή Λύσης

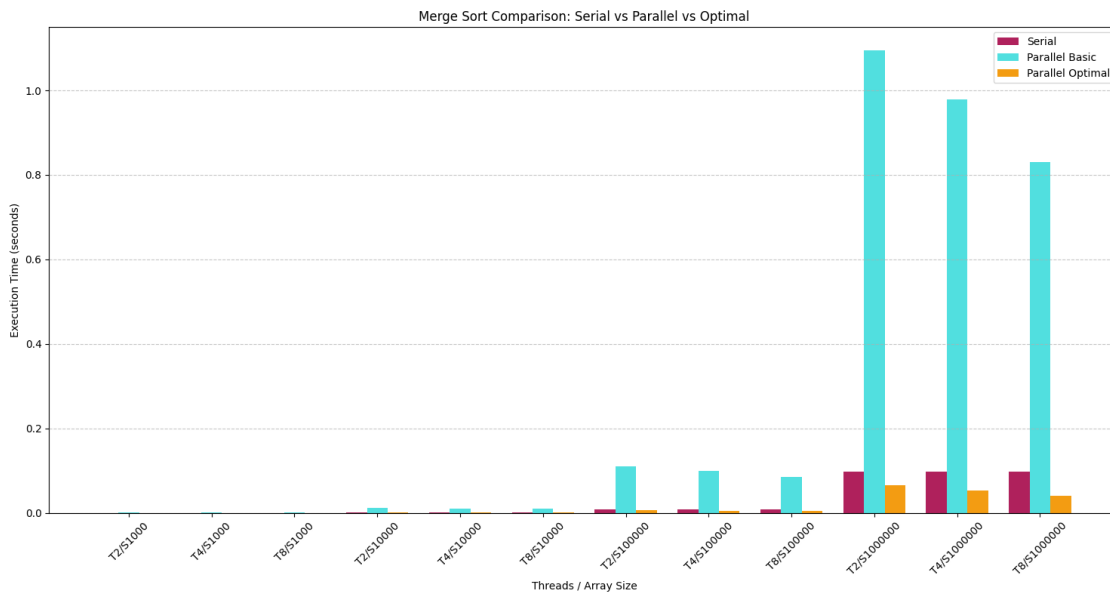
Αρχικά το πρόγραμμα δεχεται 3 ορίσματα το πλήθος των στοιχείων του πίνακα , ποιον τρόπο εκτέλεσης θα χρησιμοποιήσουμε (σειριακό ή παράλληλο) και τον αριθμό των νημάτων σε περίπτωση που θα χρησιμοποιήσουμε τον παράλληλο αλγόριθμο . Ο πίνακας θα πρέπει να τον αρχικοποιήσουμε με τυχαίες τιμές με την χρήση της συνάρτησης *rand()* .Θα πρέπει επίσης να δεμεύσουμε δυναμικά μνήμη για έναν βοηθητικό πίνακα (*temp*) ίδιου μεγεθους ο οποίος είναι απαραίτητος για τη διαδικασία της συγχώνευσης (*merge*) που θα εξηγήσουμε παρακάτω ώστε να αποφύγουμε την συνεχής δεμεύση και αποδεμεύση μνήμης κατά την αναδρομή.Τέλος μετά την υλοποίηση των αλγορίθμων θα ελεγχουμε αν ο πίνακας ταξινομηθήκε σωστά συγκρίνοντας κάθε στοιχείο με το επομενο του ώστε να επιβεβαιώσουμε την αυξουσα σειρά(το 1ο με το 2ο , το 2ο με το 3ο , ...). Τώρα θα περιγράψουμε κάθε αλγόριθμο ξεχωριστά.

Στη σειριακή εκτέλεση έχουμε δημιουργήσει τη συνάρτηση *mergesort_serial* που χωρίζει αναδρομικά τον πίνακα σε δυο μισα μέχρι να φτάσει σε υπο πίνακες με ένα στοιχείο. Στη συνέχεια η συνάρτηση *merge* συνδυάζει κάθε φορά τους πίνακες ταξινομώντας κάθε φορά συγκρίνοντας τους δείκτες των δυο τμημάτων και τοποθετώντας το μικρότερο στοιχείο στον βοηθητικό πίνακα(*temp*). Τέλος η συνάρτηση *merge* αντιγράφει πίσω στον κύριο πίνακα τα ταξινομημένα αποτελέσματα με την μεμριψ. Η λογική αυτή διασφαλίζει ότι κάθε επίπεδο της αναδρομής θα έχει ως αποτέλεσμα ένα πλήρως ταξινομημένο τμήμα και επομένως ο συνδυασμός δυο τμημάτων θα είναι επίσης ταξινομημένος .

Αντιστοίχα στην παράλληλη υλοποίηση θα χρησιμοποιήσουμε εντολές από το *OpenMP* και *tasks* το οποίο είναι ιδανικό για προγράμματα που χρησιμοποιούν την αναδρομή .Ξεκινάμε από την κλήση των εντολών `pragma omp parallel` και `pragma omp single` για να διασφαλίσουμε ότι η αρχική αναδρομική κλήση θα γίνει από ένα μόνο νήμα για να μπορούμε να αποφύγουμε τον πολλαπλασιασμό των ιδίων κλήσεων . Έχουμε δημιουργήσει τη συνάρτηση *mergesort_parallel* όπου κάθε φορά που διαιρούμε τον πίνακα η εντολή `pragma omp task` "πακταρεί" την αναδρομή του κάθε μισού πίνακα και την τοποθετεί σε μια κοινή ουρά εργασιών. Από εκεί τα διαθέσιμα νήματα επιλέγουν και υλοποιούν τις εργασίες αυτές συναμικά επιτυγχάνοντας έτσι την παράλληλη υλοποίηση . Για να διασφαλίσουμε ότι το πρόγραμμα θα λειτουργήσει σωστά θα πρέπει να χρησιμοποιήσουμε την εντολή `pragma omp taskwait` όπου θα λειτουργήσει ως σημείο συγχρονισμού των 2 αναδρομικών συναρτήσεων που καλούμε κάθε φορά για το κάθε μισό του πίνακα. Πιο συγκεκριμένα η εντολή αυτή επιβάλλει στον γονέα να περιμένει την εκτέλεση του *merge()* μέχρι να ολοκληρωθούν οι εργασίες και στα δυο τμήματα. Αυτό το κομμάτι είναι απαραίτητο καθώς θα πρέπει να δυο υπο τμήματα να ήδη ήδη πλήρως ταξινομημένα ώστε να έχουμε σωστή την τελική ταξινόμηση του αρχικού πίνακα.

Επιπλέον υλοποιήθηκε μια βελτιστοποιημένη εκδοχή της παράλληλης συνάρτησης η *mergesort_parallel_optimal* την οποία θα εξηγήσουμε στην παρακάτω ενότητα τον λόγο προσθήκης . Η βασική διαφορά από το *mergesort_parallel* είναι η προσθήκη ενός ορίου (*cutoff*) στο μέγεθος του πίνακα ($n < 1000$) με την χρήση της εντολής *if* στην οδηγία `pragma omp task` .Η λογική αυτής της βελτιστοποίησης είναι να περιορίσει τη δημιουργία υπερβολικού αριθμού εργασιών (*tasks*) για πολύ μικρά τμήματα πίνακα. Όταν το μέγεθος του υποπίνακα πέσει κάτω από 1000 ο αλγόριθμος σταματά να δημιουργεί νέα *tasks* και συνεχίζει τη διαδικασία σειριακά. Με αυτόν τον τρόπο πετυχαίνουμε καλύτερους χρόνους εκτέλεσης που θα δικαιολογήσουμε παρακάτω.

3.3 Πειραματικά Αποτελέσματα και Σχολιασμός



(α') This is a general graph with bars that shows the times of every algorithm(serial/parallel) for every thread and every array size

Αρχικά η πρώτη και η πιο βασική παρατήρηση που μπορούμε να κάνουμε με βάση το παραπάνω γραφήμα είναι ότι η χρήση της παραλληλκής υλοποίησης (*mergesort_parallel*) είναι σημαντικά πιο αργή ακόμα και από την σειριακή εκτέλεση. Αυτό συμβαίνει λόγω του τεραστίου *overhead* που δημιουργεί η *OpenMP* όταν καλούμε να διαχειριστεί χιλιάδες *tasks* που δημιουργούνται κάθε φορά που οι πίνακες χωρίζονται στην μέση. Για παράδειγμα αν έχουμε έναν πίνακα μεγέθους 10^4 δημιουργούνται περίπου 20.000 *tasks* με χρόνο 0.0116 (2 threads) παρουσιάζοντας ένα *speedup* 0.06x συγκριτικά με τον σειριακό έτσι όπως παρατηρούμε από τον πίνακα. Πιο συγκεκριμένα ο χρόνος που χρειάζεται το σύστημα για τη δημιουργία και τον προγραμματισμό αυτών των *tasks* είναι πολλαπλάσιος από τον χρόνο της ίδιας της ταξινόμησης. Επίσης επειδή τα *tasks* είναι πολύ μικρά (*size* = 1 στοιχείο) τα δεδομένα μετακινούνται συνεχώς μεταξύ της κύριας μνήμης (*RAM*) και των κρυφών μνημών (*L1/L2/L3cache*) με αποτέλεσμα η *CPU* να περιμένει τη μνήμη αντί να επεξεργάζεται. Για αυτό τον λόγο για τα μικρά μεγέθη πίνακα είναι πιο αποτελεσματικό να χρησιμοποιούμε τον σειριακό αλγόριθμο. Έτσι θα δημιουργήσουμε έναν αλγόριθμο ταξινόμησης παραλλήλου προγραμματισμού όπου είναι ο βελτιστός το *mergesort_parallel_optimal*

Table 1: Basic Parallel Mergesort Speedup

Size	Threads	Serial (s)	Parallel (s)	Speedup
1000	2	0.0001	0.0012	0.04x
1000	4	0.0001	0.0011	0.05x
1000	8	0.0001	0.0013	0.04x
10000	2	0.0007	0.0116	0.06x
10000	4	0.0007	0.0103	0.07x
10000	8	0.0007	0.0096	0.08x
100000	2	0.0086	0.1103	0.08x
100000	4	0.0086	0.1001	0.09x
100000	8	0.0086	0.0854	0.10x
1000000	2	0.0968	1.0949	0.09x
1000000	4	0.0968	0.9782	0.10x
1000000	8	0.0968	0.8297	0.12x

(α') Table with the exact times for serial and parallel algorithm with the speedup

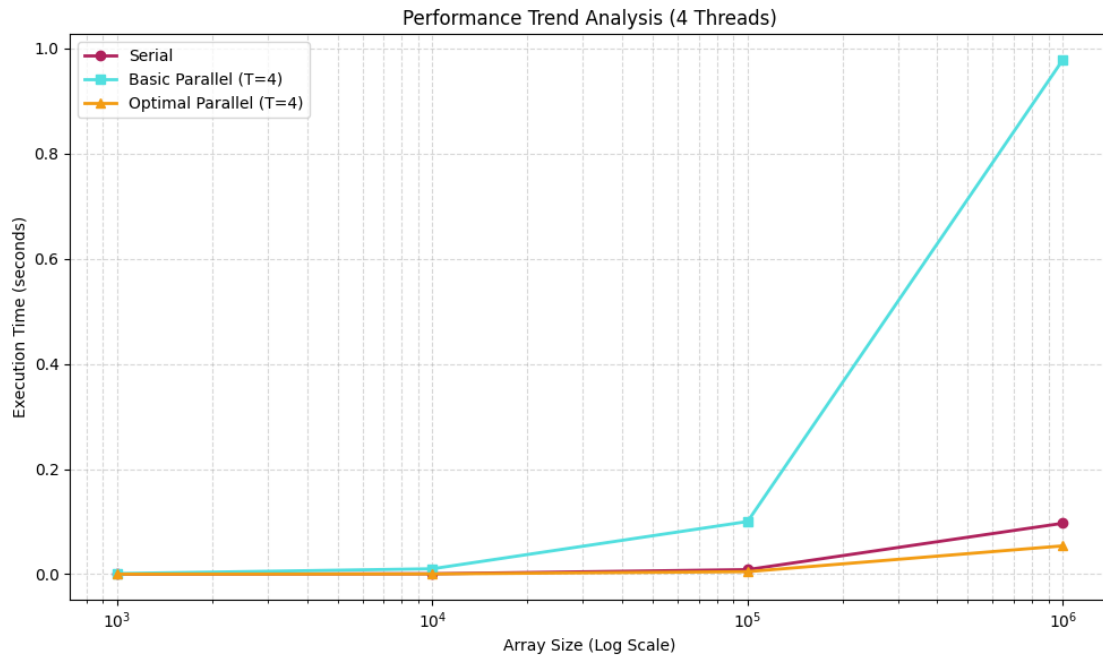
Table 2: Optimal Parallel Mergesort Speedup (Cutoff=1000)

Size	Threads	Serial (s)	Optimal (s)	Speedup
1000	2	0.0001	0.0001	0.60x
1000	4	0.0001	0.0001	0.61x
1000	8	0.0001	0.0002	0.35x
10000	2	0.0007	0.0005	1.41x
10000	4	0.0007	0.0004	1.79x
10000	8	0.0007	0.0005	1.43x
100000	2	0.0086	0.0059	1.46x
100000	4	0.0086	0.0048	1.80x
100000	8	0.0086	0.0042	2.05x
1000000	2	0.0968	0.0658	1.47x
1000000	4	0.0968	0.0538	1.80x
1000000	8	0.0968	0.0406	2.39x

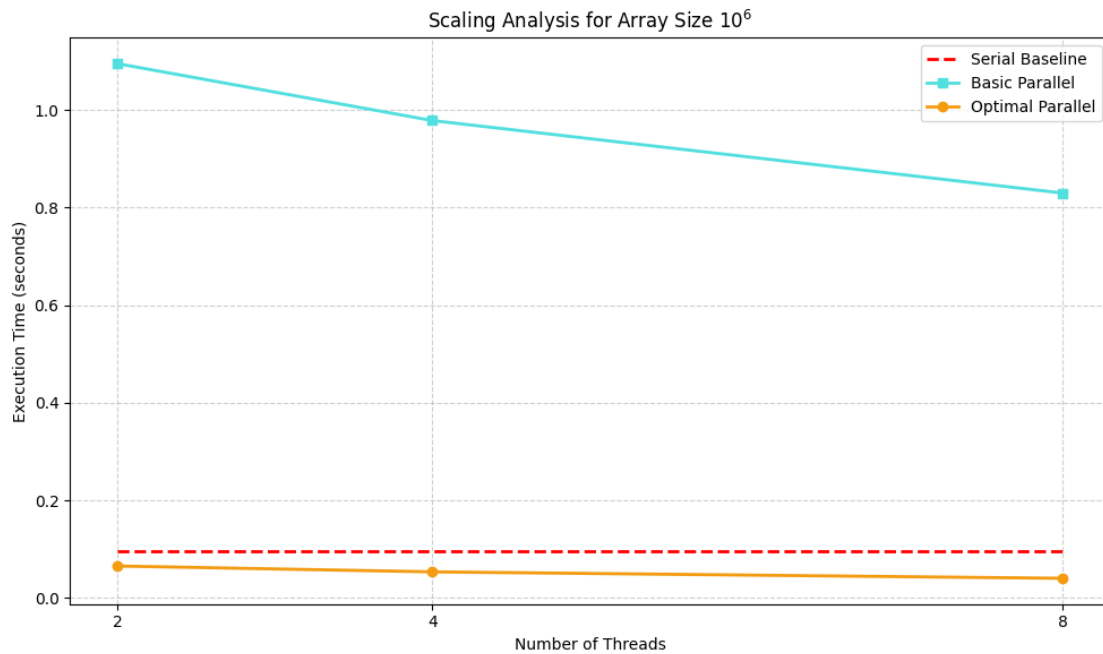
(β') Table with the exact times for serial and the optimal parallel algorithm with the speedup

Όπως βλέπουμε αυτό το πρώτο γραφήμα και τον παραπάνω πίνακα η επίδραση του *Cutoff* στην 'Optimal' είναι μεγάλη. Ουσιαστικά προσθέσαμε ένα όριο (*Cutoff* = 1000) στην *mergesort_{parallel_optimal}* η οποία καλείται για να περιορίσει τη δημιουργία νέων εργασιών όταν το μέγεθος του υποπίνακα γίνεται κάτω από το συγκεκριμένο όριο. Με αυτή τη στρατηγική ο αλγόριθμος χρησιμοποιεί τον παραλληλισμό μόνο στα ανώτερα επίπεδα της αναδρομής όπου το υπολογιστικό φορτίο είναι μεγάλο και αλλάζει σε σειριακή εκτέλεση στα χαμηλότερα επίπεδα που δεν χρειάζεται να δημιουργήσουμε τόσα πολλά *tasks*. Για να το καταλάβουμε καλύτερα χρησιμοποιώντας το παράδειγμα από τον πίνακα, για μέγεθος 10^6 με την προηγούμενη εκτέλεση

δημιουργοντουσαν περιπου 20.000 *tasks* με χρονο 0.0116 (2 *threads*) και παρουσιαζοταν ενα *speedup* 0.06x. Τωρα ομως για το 'Optimal' εχουμε χρονο 0.005 (2 *threads*) παρουσιαζοντας ενα *speedup* 1.41x . Αυτο συμβαινει διοτι δημιουργει *tasks* μονο για τα πρωτα 4 επιπεδα ($2 + 4 + 8 + 16 = 30tasks$) .Αυτο επιτρεπει στα νηματα να αφιερωνουν τον χρονο τους στην επεξεργασια των δεδομενων δηλαδη την ταξινομηση αντι για τη διαχειριση της 'γραφειοκρατιας' του παραλληλισμου. Οι πυρηνες της *CPU* ξοδεουν κυκλους ρολογιου προσπαθωντας να αποφασισουν ποιο νημα θα παρει ποιο *task* απο την ουρα εργασιων του *OpenMP*.



(α') A graph with a fixed number of threads to see the evolution of time when we increase the table size



(β') A graph with a fixed table size to see the evolution of time when we increase the number of threads

Τελος εκανα 2 επιπλεον γραφηματα που εχω θεσει εναν σταθερο αριθμο νηματων για να δουμε την επιδοση του χρονου οσο αυξανεται το μεγαθος του πινακα και εχω θεσει σταθερο το μεγαθος του πινακα για δουμε την επιδοση του χρονου οσο αυξανουμε τα *threads*. Οπως ηταν αναμενο οσο αυξανεται το μεγαθος του πινακα αυξανεται και ο χρονος. Ωστοσο καθως το μεγαθος αυξανεται προς το 10^6 η καμπυλη της απλης παραλληλης υλοποιησης (γαλαζια γραμμη) εκτοξευεται προς τα πανω επιβεβαιωνοντας οτι το *overhead* των *tasks* κυριαρχει.

Αντιθέτα όταν έχουμε σταθερό το μέγεθος του πίνακα και αυξούμε τον αριθμό νημάτων ο χρόνος μειώνεται. Είναι φανερό ότι η απλή παράλληλη υλοποίηση παρουσιάζει μεγαλύτερη βελτίωση καθώς αυξάνονται τα νήματα αλλά ο χρόνος της παραμένει σε εξαιρετικά υψηλά επίπεδα πολύ χειρότερα από τη σειριακή με βάση την γραφική παράσταση .